Data Lineage Implementation — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Data Lineage Implementation (Microsoft Purview) 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 4

Scope boundary . . . 5

Outcome . . . 5

1. Solution Overview — Data Lineage Implementation (Microsoft Purview) 5

What this solution is . . . 5

Why this matters . . . 6

How it works . . . 6

Where lineage comes from (the capture model) . . . 7

Naming convention (used in every doc and script) . . . 7

What you can control . . . 8

Limitations (be honest with buyers) . . . 8

The standard deployment model . . . 8

2. Requirements Guide — Data Lineage Implementation (Microsoft Purview) 9

2.1 A Microsoft Purview (Data Governance) account with the Data Map / Unified Catalog available . 9

2.2 Data-plane governance roles (least-privilege) on the collections in scope . . . 9

2.3 PowerShell + Azure REST toolchain (Az + data-plane token) . . . 10

2.4 In-scope data sources registered and scanned into the Data Map . . . 10

2.5 Lineage-producing connections for the automatic sources . . . 10

2.6 A custom-lineage plan for sources that don't self-report . . . 10

2.7 Data-governance stakeholders and the coverage target . . . 11

2.8 A pilot scope and change control . . . 11

Checklist . . . 11

Global Requirements — Microsoft Purview Solutions 11

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 12

2. Purview roles (least-privilege) and portal access . . . 12

3. Security & Compliance PowerShell + Graph toolchain . . . 12

4. Data sources & scope in scope . . . 13

5. Compliance, records, privacy & legal stakeholders . . . 13

6. A pilot scope and change control . . . 13

3. Implementation Guide — Data Lineage Implementation (Microsoft Purview) 14

Step 1 — Assess the Data Map (Assess) . . . 14

Step 2 — Design the coverage/connection map (Design) . . . 14

Step 3 — Ensure the endpoints are scanned into the Data Map (Configure) . . . 15
Step 4 — Connect the automatic-lineage sources (Configure, pilot-first) . . . 15
Step 5 — Push custom lineage for non-automatic paths (Configure, pilot-first) . . . 15
Step 6 — Validate the pilot lineage graph (Pilot → Validate) . . . 16
Step 7 — Measure coverage (Validate) . . . 16
Step 8 — Expand across the estate (Expand) . . . 16
5. Testing Guide — Data Lineage Implementation (Microsoft Purview) 17
Test 1 — Data-plane access and account reachability . . . 17
Test 2 — Source + scan + lineage-capability inventory is complete . . . 17
Test 3 — Endpoints are scanned (assets exist as lineage endpoints) . . . 17
Test 4 — Custom-lineage dry-run makes no changes . . . 17
Test 5 — Automatic lineage appears after a pilot pipeline run . . . 18
Test 6 — Custom lineage renders after -Commit . . . 18
Test 7 — Lineage is correct, not just present . . . 18
Test 8 — Coverage report reflects the estate . . . 18
Pass/fail gate . . . 18
6. Troubleshooting Guide — Data Lineage Implementation (Microsoft Purview) 19
The Data Map / lineage capability isn't there, or the account won't provision . . . 19
`Get-AzAccessToken` / data-plane calls return 401 / 403 . . . 19
A REST call returns 404 / "api-version not supported" . . . 19
Lineage isn't appearing after connecting ADF/Synapse/Power BI . . . 19
Custom-lineage push flags an asset as "not resolvable" . . . 20
`DGLINE-Configure.ps1 -Commit` created lineage in the wrong place . . . 20
Lineage renders but points at the wrong source ("confident wrong answer") . . . 20
Coverage report shows many assets "without lineage" . . . 20
Scan shows failed / no runs for a source . . . 20
7. Best Practices — Data Lineage Implementation (Microsoft Purview) 21
8. Deliverables Summary — Data Lineage Implementation (Microsoft Purview) 22
Documentation . . . 22
PowerShell scripts . . . 23
What the buyer receives . . . 23
Outcome . . . 23
Appendix A — PowerShell Scripts 25

Executive Summary — Data Lineage Implementation (Microsoft Purview)

"Where did this number come from, and what breaks if we change it?" is a question most data programs cannot answer with evidence. Without **data lineage**, nobody can trace a report back to its sources or forward to its consumers — which is both a compliance exposure and an operational risk every time a source changes or a data-quality issue surfaces. This solution implements **data lineage in Microsoft Purview**, end to end, so that for the in-scope estate data can be traced **source → report and back**, defensibly.

It gives you a repeatable way to stand lineage up on the **Microsoft Purview Data Map / Unified Catalog**: register and scan the in-scope sources so assets exist as lineage endpoints, **connect the integration systems** (Azure Data Factory, Synapse, Data Share, Power BI) that report lineage automatically, **push custom lineage** through the Apache Atlas–compatible catalog API for the systems that don't self-report, and **visualize and validate** the resulting graph — using only the Purview portal and PowerShell/REST against the documented Azure Purview data-plane APIs, with no credentials stored in code.

This is a complete, deploy-it-today package for implementing data lineage, written for a competent Azure/Purview data-governance administrator.

What you get

• A plain-language Solution Overview of lineage in the Purview Data Map, the capture model (automatic

connectors vs. custom Atlas lineage), and the phased approach.

• A Requirements Guide with verify / implement-if-missing for every prerequisite — the Purview Data

Governance account, least-privilege data-plane roles, the Az + REST toolchain, and registered/scanned sources.

• An Implementation Guide of precise, ordered portal + PowerShell/REST steps, with lineage connected and

pushed **pilot-first** and every asset named by qualified name.

• Five PowerShell scripts: source/scan/lineage-capability discovery, a custom-lineage push helper

(`-WhatIf` by default), a lineage-graph validator, a coverage report, and a scope/health helper.

• Testing and Troubleshooting guides for a confident, safe rollout.

What this delivers

In-scope sources registered and scanned into the Data Map, automatic lineage flowing from the connected integration systems, custom lineage pushed for the systems that don't self-report, the graph validated for **correctness** (not just presence) on a pilot scope, and **coverage measured** against the governance target so gaps are explicit and closed deliberately.

The deployment model

Assess → Design → Configure → Pilot → Validate → Expand.

Scope boundary

This solution delivers **lineage** on the Data Map foundation. Building the classification/label program, DLP, retention/records, insider risk, and communication compliance are each the subject of a dedicated Purview solution and are not duplicated here. Data lineage is a **Data Governance (Data Map)** capability — separately provisioned and Azure-metered, distinct from Microsoft 365 E5 compliance; verify the current provisioning model, pricing, roles, and portal paths against Microsoft's documentation and your own tenant. What lineage coverage is *required* for a regulatory/records obligation, and which source is *authoritative* for a data element, are deferred to the data-governance, records, privacy, and legal functions — this solution provides the technical capability, the graph, and the coverage evidence, not the determination.

Outcome

Data lineage implemented in Microsoft Purview the right way — sources scanned, integration systems connected, custom lineage pushed where needed, the graph validated and coverage measured — so your organization can trace data from source to report and back, and answer "what breaks if we change this?" with a graph instead of a guess.

1. Solution Overview — Data Lineage Implementation (Microsoft Purview)

What this solution is

"Where did this number come from?" is the question a data governance program has to be able to answer. This solution implements **data lineage in Microsoft Purview** — end to end — so that for a given asset (a report, a table, a file) you can trace it **back to its sources** and **forward to where it is consumed**, across the ingestion and transformation systems that move data through the estate.

Lineage in Microsoft Purview is a capability of the **Microsoft Purview Data Map / Unified Catalog** (the data-governance side of Purview, historically "Azure Purview"). Purview builds a lineage graph by:

- **Scanning** registered data sources into the **Data Map** so the assets exist as catalog entities.
- **Automatically capturing lineage** reported by supported data-integration systems — **Azure Data Factory**, Azure Synapse pipelines**,** Azure Data Share**, and** Power BI** — as those systems run copy/transform activities and push lineage to Purview.
- **Accepting custom lineage** pushed through the **Apache Atlas–compatible catalog REST API** for systems that don't report automatically (custom ETL, Spark/Hive via connectors, home-grown pipelines).
- **Visualizing** the result on each asset's **Lineage** tab in the Purview catalog — source → process → sink, hop by hop.

Everything is done with the **Microsoft Purview portal**, the **Purview Data Map / Unified Catalog**, and **PowerShell/REST** against the documented **Azure Purview data-plane APIs** (catalog/Atlas, scanning, collections) using `Connect-AzAccount` **/** `Get-AzAccessToken` — no third-party tooling and no credentials stored in code.

> ***Scope boundary — this is lineage, built on the Data Map foundation.*** *This solution configures and validates* ***lineage capture and visualization****. It assumes (or stands up just enough of) a Purview* ***Data Map / Unified Catalog*** *with the in-scope sources* ***registered and scanned*** *so assets exist to draw lineage between. Deep classification/label programs, DLP, retention/records, insider risk, and communication compliance are the subject of the* ***other Purview solutions*** *and are not duplicated here.*

Why this matters

• **Traceability is the governance gap.** Without lineage, nobody can prove where a figure came from, which reports break if a source changes, or which downstream consumers a data-quality issue touches. That is both a compliance exposure and an operational risk.

• **Impact analysis becomes possible.** Lineage turns "if we change this table, what breaks?" from a guess into a graph you can read — upstream dependencies and downstream consumers, visible.

• **Automatic lineage is only automatic if the source is connected the right way.** ADF, Synapse, and Power BI report lineage to Purview **only** when connected and running against registered assets; getting that connection right is most of the work, and it is easy to get subtly wrong.

• **Gaps must be found, not assumed.** A lineage graph with silent holes is worse than none — you trust it and it's wrong. This solution measures **coverage** so the gaps are explicit and closed deliberately (custom lineage, additional scans) rather than discovered during an incident.

How it works

1. **Assess.** Read the Purview account, its **collections**, the **registered data sources**, and the **scan** status — and identify which sources produce lineage automatically (ADF/Synapse/Power BI) vs. which will need **custom lineage**. Read-only, no changes.

1. **Design.** Map the **lineage coverage** you need: for each in-scope pipeline/source→report path, decide how lineage is produced (automatic via a connection, or pushed via the Atlas API), and pick a **pilot scope** (one collection / a couple of representative sources) to prove it on first.

1. **Configure.** Register the lineage-producing **connections** (e.g. connect ADF/Synapse/Power BI to Purview), ensure the underlying assets are **scanned** into the Data Map, and — for sources that don't self-report — push **custom lineage** through the catalog Atlas API. Writes are previewed with `-WhatIf` before `-Commit`/`-Apply`.

1. **Pilot (test scope).** Prove lineage on the **pilot scope first**: run the pilot pipeline (or push the pilot custom lineage), then confirm the lineage graph renders correctly for the pilot assets before touching the rest of the estate. High-impact steps (connecting production sources, pushing lineage) are always exercised on the pilot scope before expansion.

1. **Validate.** Query the lineage graph for named assets through the Atlas lineage endpoint, confirm

the **upstream/downstream edges** exist and are correct, and measure **coverage** (assets with lineage vs. without).

1. **Expand.** Extend the same pattern to the remaining collections/sources, re-measuring coverage and closing gaps until the in-scope estate is traceable.

Where lineage comes from (the capture model)

Source system	How lineage is captured	What you configure
Azure Data Factory	Automatic — copy/data-flow activities report lineage to Purview	Connect the ADF account to Purview (Purview account → **Data Map** → **Integration / connections**, verify path)
Azure Synapse pipelines	Automatic — pipeline activities report lineage	Connect the Synapse workspace to Purview
Power BI	Automatic — datasets/reports report lineage on scan	Register + scan the Power BI tenant/workspaces into the Data Map
Azure Data Share	Automatic — share snapshots report lineage	Connect Data Share to Purview
Custom ETL / Spark / Hive / home-grown	**Custom lineage** pushed via the **Atlas catalog REST API** (`/catalog/api/atlas/v2/…`)	Push `Process` entities with `inputs`/`outputs` referencing scanned assets

> ***Lineage needs the endpoints to exist.*** *An automatic-lineage edge only renders if **both** ends — the source asset and the sink asset — are registered/scanned into the Data Map (or referenced by qualified name). "Connect the pipeline" and "scan the stores" go together; one without the other produces a half-drawn graph.*

Naming convention (used in every doc and script)

Artifacts follow `<Purpose> – DGLINE – <Stage>` and are reused verbatim across the docs, scripts, and portal.

Artifact	Name
Source + scan + lineage-capability inventory	`Data Lineage Source Inventory – DGLINE – Discovery`
Coverage/connection design (per source→report path)	`Data Lineage Coverage Map – DGLINE – Design`
Registered lineage connections + custom-lineage config	`Data Lineage Connections – DGLINE – Configure`
Lineage graph validation (upstream/downstream edges)	`Data Lineage Validation – DGLINE – Validate`

Artifact	Name
Lineage coverage report (with/without lineage)	`Data Lineage Coverage – DGLINE – Report`
Pilot scope (collection / pilot sources)	`Data Lineage Pilot Scope – DGLINE – Stage1`

What you can control

- Which **collections** and **data sources** are in scope for lineage.
- How lineage is produced per path — **automatic connection** vs. **custom lineage** via the Atlas API.
- The **pilot scope** used to prove lineage before expanding to the estate.
- The **coverage target** (which source→report paths must be traceable) — the *business* target is set with the data owners; you deliver the technical capability and the measured coverage against it.

Limitations (be honest with buyers)

- **Purview Data Governance is separately provisioned and billed.** Data lineage lives in the **Data Map / Unified Catalog**, which is an** Azure-metered / consumption-based **service (capacity units / pay-as-you-go under an Azure subscription) —** distinct **from the Microsoft 365 E5 / E5 Compliance capabilities. It requires a** Purview (Data Governance) account **and appropriate** data-plane roles **(Collection admin / Data source admin / Data curator).** Verify the current provisioning model, pricing, and role model against Microsoft's documentation and your own tenant** — this area is actively evolving (the classic Data Map and the new Unified Catalog experience).
- **Automatic lineage depends on the source and on Microsoft's connectors.** Only supported systems (ADF, Synapse, Data Share, Power BI, and Atlas-hook connectors) report lineage automatically, and **only for supported activities/asset types**. Coverage for unsupported systems is achieved with **custom lineage** via the Atlas API — real work, not automatic.
- **Lineage is not retroactive for runs before the connection existed.** Automatic lineage is captured as pipelines **run after** they are connected to Purview; historical runs are not back-filled. Connect early.
- **API versions and portal paths evolve.** The data-plane REST `api-version` values and portal navigation change over time and differ across the classic vs. unified experience — every one is **cited as of authoring and flagged "verify in your tenant."** No endpoint or path is invented.
- **Governance determinations are deferred.** What lineage coverage is *required* for a given regulatory/records obligation, and what the authoritative source of a data element *is*, are decisions for the **data-governance, records, privacy, and compliance** functions — this solution provides the technical capability, the graph, and the coverage evidence, not the determination.

The standard deployment model

> *Assess the Data Map (sources, scans, lineage capability) → design the coverage/connection map → configure connections + scans + custom lineage (preview with `-WhatIf` first) → pilot on a small scope*

*→ validate the graph and coverage → expand across the estate. Pattern: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.***

2. Requirements Guide — Data Lineage Implementation (Microsoft Purview)

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-purview.md` (the six shared requirements: compliance/governance licensing, least-privilege Purview roles, the PowerShell + Graph/REST toolchain, in-scope data sources, compliance/records/privacy/legal stakeholders, and a pilot scope with change control) — those are referenced here, not repeated.

> ***Read this first.** Data lineage is a **Microsoft Purview Data Map / Unified Catalog** capability (the data-governance side), **not** a Microsoft 365 E5 compliance feature. Its prerequisites are a **Purview (Data Governance) account**, **data-plane roles**, and **connected + scanned data sources** — different from the compliance-side prerequisites. Everything below is **cited as of authoring and flagged "verify in your tenant"**; provisioning, pricing, roles, portal paths, and REST `api-version` values in this area are actively evolving. Nothing is invented.*

2.1 A Microsoft Purview (Data Governance) account with the Data Map / Unified Catalog available

Description: A provisioned **Microsoft Purview account** (Data Map / Unified Catalog) under an Azure subscription, reachable at its data-plane endpoint `https://<account>.purview.azure.com`. **Why:** Lineage is a Data Map capability — assets, scans, and the lineage graph live in this account. Without it there is nothing to draw lineage in (extends Global Requirement 1 to the governance side). **Verify:** In the **Purview portal** confirm the account/catalog loads and the **Data Map** is present; `DGLINE-ScopeHealth.ps1` confirms the account endpoint is reachable and a data-plane token is obtainable. **Implement:** Provision a Purview (Data Governance) account (Azure portal / `Az.Purview`) under change control. **Verify the current provisioning model and pricing** (Data Map is Azure-metered / consumption-based — capacity units / pay-as-you-go) against Microsoft's documentation before creating it.

2.2 Data-plane governance roles (least-privilege) on the collections in scope

Description: The **least-privilege Purview data-plane roles** the work needs, assigned **at the collection scope**: **Collection admin** (to register sources / manage collections), **Data source admin** (to register/scan sources and manage credentials), and **Data curator** (to read the catalog, edit assets, and read/edit lineage). Reserve broad/root-collection admin for the few setup actions (Global Requirement 2). **Why:** Purview Data Map uses **collection-scoped** roles distinct from the M365 compliance role groups; registering sources, running scans, and editing lineage each require the matching data-plane role at the right collection. **Verify:** Purview portal → **Data Map** → **Collections** → **<collection>** → **Role assignments** (verify path) shows your account in the required roles; `DGLINE-ScopeHealth.ps1` reports the collections your token can enumerate. **Implement:** Assign the least-privilege data-plane roles at the pilot collection first, root-collection admin only where genuinely required, under change control.

2.3 PowerShell + Azure REST toolchain (Az + data-plane token)

Description: A workstation with the **Az PowerShell** modules (`Az.Accounts`, and `Az.Purview` for control-plane account operations), a modern-auth/MFA admin, and the ability to acquire a **Purview data-plane token** via `Get-AzAccessToken -ResourceUrl 'https://purview.azure.net'` for `Invoke-RestMethod` calls to the catalog/scanning APIs (Global Requirement 3). **Why:** The provided scripts read and (with `-Commit`) configure lineage through the documented Purview data-plane REST APIs, authenticated with an Azure AD token — no credentials in code. **Verify:** `Get-Module -ListAvailable Az.Accounts, Az.Purview`; `Connect-AzAccount` succeeds and `Get-AzAccessToken -ResourceUrl 'https://purview.azure.net'` returns a token. `DGLINE-ScopeHealth.ps1` confirms connectivity. **Implement:** `Install-Module Az.Accounts, Az.Purview -Scope CurrentUser`; sign in with a modern-auth admin; **verify the current data-plane resource URL /** `api-version` **values** in Microsoft's docs.

2.4 In-scope data sources registered and scanned into the Data Map

Description: The **data sources** that make up each in-scope source→report path (e.g. Azure Data Lake / Blob, Azure SQL / Synapse SQL, Power BI) **registered** to a collection and **scanned**, so the assets exist as catalog entities to be lineage endpoints (Global Requirement 4). **Why:** A lineage edge only renders when **both** ends are known to the Data Map. Un-scanned sources produce half-drawn or missing lineage. **Verify:** Purview portal → **Data Map** → **Data sources** shows the sources registered with **completed** scans; `DGLINE-Discovery.ps1` lists registered sources and last-scan status. Assets are searchable in the catalog. **Implement:** Register the sources to the in-scope collection and run (or schedule) scans; confirm the assets appear before expecting lineage between them.

2.5 Lineage-producing connections for the automatic sources

Description: The **connections** that make ADF / Synapse / Data Share / Power BI report lineage to Purview — the ADF/Synapse workspace connected to this Purview account, and the Power BI tenant/workspaces registered and scanned. **Why:** Automatic lineage is produced **only** when the integration system is connected to Purview and runs against registered assets; the connection is the switch that turns lineage on. **Verify:** Purview portal → **Data Map** → **Integration runtimes / connections** (verify path) shows the ADF/Synapse connection; a pipeline run subsequently shows a lineage edge on the affected asset. `DGLINE-Discovery.ps1` flags which in-scope sources are lineage-capable and connected. **Implement:** Connect ADF/Synapse to Purview and register/scan Power BI, per the capture-model table in the Overview; verify the current connection path in your tenant (classic vs. unified experience differ).

2.6 A custom-lineage plan for sources that don't self-report

Description: For pipelines/systems **not** covered by an automatic connector (custom ETL, Spark/Hive without a supported hook, home-grown movers), a plan to push **custom lineage** via the **Atlas catalog REST API** (`Process` entities with `inputs`/`outputs` referencing scanned assets by qualified name). **Why:** Coverage of unsupported systems is achieved with custom lineage — it does not appear on its own; this is deliberate work, scoped in the Design step. **Verify:** The list of non-automatic paths is recorded in `Data Lineage Coverage Map – DGLINE – Design`; `DGLINE-Configure.ps1 -WhatIf` shows the `Process` entity it would push for a pilot path. **Implement:** Identify the qualified names of the

source/sink assets and author the custom-lineage `Process` payloads; push them (pilot first) with `DGLINE-Configure.ps1`.

2.7 Data-governance stakeholders and the coverage target

Description: Engagement with the **data-governance, records, privacy, and business-data owners** (Global Requirement 5) who define **which source→report paths must be traceable** (the coverage target) and **which source is authoritative** for a data element. **Why:** The lineage program aligns to governance requirements, not technical convenience; what "fully traceable" means, and which regulatory/records obligation drives it, is the stakeholders' determination. **Verify:** Named owners and an agreed coverage target (the in-scope paths) are recorded. **Implement:** Identify the owners and capture the coverage target before configuring; you deliver the capability and the measured coverage, they own the target and the determination.

2.8 A pilot scope and change control

Description: A **pilot scope** (`Data Lineage Pilot Scope – DGLINE – Stage1`) — one collection or a couple of representative sources / one source→report path — and a **change-control** process, per Global Requirement 6. **Why:** Connecting production integration sources, running scans, and pushing lineage are high-impact; they are proven on the pilot scope before the estate. The model is **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**. **Verify:** The pilot scope is named and a change window/approval path exists. **Implement:** Define the pilot scope and agree change control before connecting production sources or pushing lineage tenant-wide.

Checklist

- Microsoft Purview (Data Governance) account provisioned; Data Map / Unified Catalog available (pricing/model verified)
- Data-plane roles (Collection admin / Data source admin / Data curator) assigned least-privilege at the in-scope collections
- `Az.Accounts` + `Az.Purview` installed; `Connect-AzAccount` and `Get-AzAccessToken` (purview.azure.net) succeed
- In-scope sources registered **and scanned** into the Data Map (assets exist as lineage endpoints)
- Lineage-producing connections in place for automatic sources (ADF / Synapse / Data Share / Power BI)
- Custom-lineage plan recorded for non-automatic paths (Atlas `Process` entities)
- Data-governance/business owners named; coverage target (in-scope paths) agreed
- `Data Lineage Pilot Scope – DGLINE – Stage1` defined; change control agreed

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication

compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

- **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

- **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

- **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

- **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

- **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

- **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.
- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.

- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams, endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.
- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with stakeholders.
- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal** stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).
- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements, not technical defaults; several capabilities have HR/legal sensitivity.
- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.
- **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate → Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.
- **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change window/approval path.
- **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant

governance functions.*

3. Implementation Guide — Data Lineage Implementation (Microsoft Purview)

Follow these steps **in order**. Work is done in the **Microsoft Purview portal**, the **Data Map / Unified Catalog**, and **PowerShell/REST** against the documented **Azure Purview data-plane APIs** (catalog/Atlas, scanning, collections) authenticated with `Connect-AzAccount` / `Get-AzAccessToken -ResourceUrl 'https://purview.azure.net'`. All writes are previewed with `-WhatIf` before `-Commit`/`-Apply`. Artifact names follow the `<Purpose> – DGLINE – <Stage>` convention from the Overview and are reused verbatim.

> ***Model:*** *Assess the Data Map, design the coverage/connection map, configure connections + scans + custom lineage, prove it on a* ***pilot scope****, validate the graph and coverage, then expand across the estate. Pattern:* ***Assess*** *→* ***Design*** *→* ***Configure*** *→* ***Pilot*** *→* ***Validate*** *→* ***Expand. Portal paths and REST*** `api-version` ***values are cited as of authoring and this area is actively evolving (classic Data Map vs. the unified Unified Catalog experience) — verify each in your own tenant.*** *Data lineage is a* ***Data Governance (Data Map)*** *capability, separately provisioned and billed from E5 compliance (Requirement 2.1). Automatic lineage is captured only for* ***runs after*** *the connection exists — connect early; history is not back-filled.*

Step 1 — Assess the Data Map (Assess)

1. Open the **Purview portal** and confirm the account, the **Data Map**, and the **Collections** tree

load. Note the account data-plane endpoint `https://<account>.purview.azure.com`.

1. Run `DGLINE-ScopeHealth.ps1 -Account <account>`. Read-only, it confirms the account endpoint is

reachable, acquires a **data-plane token** (`purview.azure.net`), and lists the **collections** your token can enumerate — establishing you have the data-plane roles to proceed.

1. Run `DGLINE-Discovery.ps1 -Account <account> -CsvPath .\LineageDiscovery.csv`. Read-only, it captures:

the **registered data sources**, their **last-scan status**, and — per source — whether it is **lineage-capable/connected** (ADF/Synapse/Data Share/Power BI report automatically) or will need **custom lineage**. It writes `Data Lineage Source Inventory – DGLINE – Discovery`.

1. **Pass/fail:** you have a complete inventory — for every in-scope source you can state "registered + scanned" and "automatic lineage / needs custom lineage".

Step 2 — Design the coverage/connection map (Design)

1. From the discovery output and the stakeholders' **coverage target** (Requirement 2.7), build

`Data Lineage Coverage Map – DGLINE – Design`: for each in-scope **source→report path**, record the assets at each hop (by qualified name), and **how lineage is produced** — an automatic connection (ADF/Synapse/Data Share/Power BI) or a **custom** `Process` pushed via the Atlas API.

1. Confirm each path's endpoints are (or will be) **registered and scanned** into the Data Map — a lineage edge needs both ends known (Requirement 2.4). Flag any endpoint not yet scanned as a prerequisite task.

1. Pick the **pilot scope** `Data Lineage Pilot Scope – DGLINE – Stage1`: one collection or a single representative source→report path (ideally one automatic and one custom, to prove both mechanisms).

1. **Pass/fail:** every in-scope path has a defined lineage-production method and confirmed endpoints; a pilot scope is named.

Step 3 — Ensure the endpoints are scanned into the Data Map (Configure)

1. For each path endpoint not yet scanned, register the source to its collection and run a scan (Purview portal → **Data Map** → **Data sources** → **Register / New scan**, verify path). Re-run `DGLINE-Discovery.ps1` and confirm **completed** scans.

1. Confirm the assets are searchable in the catalog (portal search, or the catalog search API surfaced by `DGLINE-Validate.ps1`).

1. **Pass/fail + rollback:** every in-scope endpoint shows a completed scan and resolvable assets.

Rollback for a mis-registered source is to **remove the scan/source** in the portal — no lineage has been drawn yet.

Step 4 — Connect the automatic-lineage sources (Configure, pilot-first)

1. For the pilot path's automatic source, connect the integration system to **this Purview account**: ADF/Synapse workspace → connect to Purview; Power BI → register + scan the tenant/workspaces (Purview portal → **Data Map** → **Integration / connections**, verify path — classic vs. unified differ).

1. **Run the pilot pipeline** (or refresh the Power BI dataset) so the activity **reports lineage**. Automatic lineage is captured on the **run after** the connection exists — a pre-existing run is not back-filled.

1. Re-run `DGLINE-Discovery.ps1` to confirm the connection is recognized. Record the connection in `Data Lineage Connections – DGLINE – Configure`.

1. **Pass/fail + rollback:** the pilot pipeline run produces a lineage edge (validated in Step 6).

Rollback is to **disconnect** the integration source from Purview in the portal; no source data is altered.

Step 5 — Push custom lineage for non-automatic paths (Configure, pilot-first)

1. For the pilot's custom path, dry-run first: `DGLINE-Configure.ps1 -Account <account> -MapPath .\LineageMap.csv -WhatIf`. With `-WhatIf` (the default) it makes **no changes** — it prints the **Atlas** `Process` **entity** it would push (the `inputs`/`outputs` qualified names, the process type/name) and flags any referenced asset it cannot

resolve in the catalog.

1. Read every line against `Data Lineage Coverage Map – DGLINE – Design`. Confirm the input/output

qualified names resolve to the intended scanned assets and no unintended entity would be created.

1. Apply to the **pilot path only**: `DGLINE-Configure.ps1 -Account <account> -MapPath .\LineageMap.csv

-Commit`. It POSTs the Process entity to the catalog /catalog/api/atlas/v2/entity (or /lineage) endpoint (verify api-version), creating the lineage relationship. Updates Data Lineage Connections – DGLINE – Configure`.

1. **Pass/fail + rollback:** the pilot custom-lineage `Process` is created and renders (Step 6). Rollback

is to **delete the pushed** `Process` **entity** by GUID (the script prints the created GUID and the exact reverse `DELETE` call) — only the lineage entity is removed, never the underlying data assets.

Step 6 — Validate the pilot lineage graph (Pilot → Validate)

1. Run `DGLINE-Validate.ps1 -Account <account> -Asset '<qualified-name>'` for a pilot sink asset.

Read-only, it queries the **Atlas lineage endpoint** (`/catalog/api/atlas/v2/lineage/`…, verify `api-version`) and reports the **upstream** and **downstream** entities and the edge count.

1. In the portal, open the pilot asset's **Lineage** tab and confirm the graph matches — source →

process → sink, correct hops.

1. Have a data owner confirm the pilot path's lineage is **correct** (right sources, right consumers) —

not just present. **Pass/fail:** the pilot path's lineage renders and is confirmed accurate by the owner; complete the relevant **Testing Guide** gates on the pilot scope before expanding.

Step 7 — Measure coverage (Validate)

1. Run `DGLINE-Report.ps1 -Account <account> -CsvPath .\LineageCoverage.csv`. Read-only, it enumerates

the in-scope assets and reports **coverage** — assets **with** lineage vs. **without**, per collection / source — and the last-scan status. It writes `Data Lineage Coverage – DGLINE – Report`.

1. Compare coverage against the stakeholders' target (Requirement 2.7). Every gap is an explicit

follow-up (a missing scan, a missing connection, or a custom-lineage `Process` to push) — **not** a silent hole.

1. **Pass/fail:** a dated coverage report exists; each gap is recorded as a named follow-up item.

Step 8 — Expand across the estate (Expand)

1. Repeat Steps 3–7 for the remaining in-scope collections/sources: scan the endpoints, connect the

automatic sources (`-WhatIf` → run the pipeline), push custom lineage (`-WhatIf` → `-Commit`), then validate and re-measure coverage.

1. Re-run `DGLINE-Report.ps1` after each wave and drive coverage toward the target, closing gaps deliberately.

1. **Determinations about what coverage is *required* (regulatory/records) and which source is authoritative remain the governance team's **— you deliver the capability, the graph, and the measured coverage. Hand off any classification/label/DLP/retention work to the dedicated Purview solutions; this solution delivers** lineage**.

Order of operations

`Assess Data Map (sources, scans, lineage capability) → Design coverage/connection map + pilot scope → Scan endpoints into the Data Map → Connect automatic sources (pilot; run the pipeline) → Push custom lineage (-WhatIf → -Commit, pilot) → Validate the pilot graph → Measure coverage → Expand across the estate, re-measuring coverage`

5. Testing Guide — Data Lineage Implementation (Microsoft Purview)

Run after each configuration step. Every test is safe (read-only or a `-WhatIf` preview). Validate on the `Data Lineage Pilot Scope – DGLINE – Stage1` before expanding to the estate.

Test 1 — Data-plane access and account reachability

Do: Run `DGLINE-ScopeHealth.ps1 -Account <account>`. **Expected:** The account endpoint `https://<account>.purview.azure.com` is reachable, a `purview.azure.net` data-plane token is acquired, and the collections you have roles on are listed — confirming the least-privilege data-plane roles are in place.

Test 2 — Source + scan + lineage-capability inventory is complete

Do: Run `DGLINE-Discovery.ps1 -Account <account>`; compare against **Data Map** → **Data sources** in the portal. **Expected:** Every in-scope source is listed with its **last-scan status**, and each is correctly flagged **automatic lineage (connected)** or **needs custom lineage** — matching what the portal shows.

Test 3 — Endpoints are scanned (assets exist as lineage endpoints)

Do: For a pilot path, search the pilot source/sink assets in the portal catalog (or via `DGLINE-Validate.ps1`, which resolves them by qualified name). **Expected:** Both endpoints resolve to scanned catalog assets with a **completed** scan — the prerequisite for any lineage edge between them is satisfied.

Test 4 — Custom-lineage dry-run makes no changes

Do: Run `DGLINE-Configure.ps1 -Account <account> -MapPath .\LineageMap.csv -WhatIf`; afterward query the pilot sink asset's lineage with `DGLINE-Validate.ps1`. **Expected:** The dry-run prints the exact **Atlas** `Process` it would push (inputs/outputs qualified names) and flags any unresolved asset; the lineage graph is **unchanged** — `-WhatIf` truly changed nothing.

Test 5 — Automatic lineage appears after a pilot pipeline run

Do: With the pilot ADF/Synapse source connected to Purview, run the pilot pipeline, then run `DGLINE-Validate.ps1 -Account <account> -Asset '<pilot sink qualified name>'`. **Expected:** The lineage query returns an **upstream** edge from the pilot source (produced by the pipeline run), and the asset's **Lineage** tab in the portal renders source → process → sink. (Allow for processing delay after the run.)

Test 6 — Custom lineage renders after -Commit

Do: After `DGLINE-Configure.ps1 … -Commit` on the pilot custom path, run `DGLINE-Validate.ps1 -Account <account> -Asset '<pilot sink qualified name>'` and open the Lineage tab. **Expected:** The pushed `Process` entity appears as a lineage node with the intended inputs/outputs; the graph matches `Data Lineage Coverage Map – DGLINE – Design`. The script reported the created GUID (for rollback).

Test 7 — Lineage is correct, not just present

Do: With a data owner, walk the pilot path's lineage graph end to end. **Expected:** The upstream sources and downstream consumers are **accurate** (right authoritative source, no missing or spurious hops) — confirmed by the owner, not just rendered.

Test 8 — Coverage report reflects the estate

Do: Run `DGLINE-Report.ps1 -Account <account> -CsvPath .\LineageCoverage.csv`; review `Data Lineage Coverage – DGLINE – Report`. **Expected:** The report accurately splits in-scope assets into **with-lineage** vs. **without-lineage** per collection/source, and each without-lineage item corresponds to a real gap (missing scan, missing connection, or a custom-lineage `Process` still to push) recorded as a follow-up.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–7 pass on `Data Lineage Pilot Scope – DGLINE – Stage1`: data-plane access confirmed, inventory complete, endpoints scanned, `-WhatIf` inert, automatic + custom lineage render, and the pilot graph is confirmed **accurate** by an owner

Gate	Requirement
Expand pass	Test 8 passes across each expansion wave — coverage measured against the target, every gap recorded as a named follow-up — with automatic/custom lineage continuing to render as sources are added

6. Troubleshooting Guide — Data Lineage Implementation (Microsoft Purview)

Field issues and how to resolve them, in the order you are likely to hit them. Every check here is safe (read-only or a `-WhatIf` preview).

The Data Map / lineage capability isn't there, or the account won't provision

Symptom: No Data Map in the portal, or you can't create a Purview (Data Governance) account, or lineage features are absent. **Cause:** Data lineage is a **Data Governance (Data Map / Unified Catalog)** capability — separately provisioned and **Azure-metered** (capacity units / consumption), **distinct** from Microsoft 365 E5 / E5 Compliance. Without a provisioned Purview account there is nothing to draw lineage in; and the classic Data Map vs. the newer unified experience differ. **Fix:** Provision a Purview (Data Governance) account under an Azure subscription. **Verify the current provisioning model, pricing, and which experience your tenant is on** against Microsoft's documentation — this area is actively evolving. `DGLINE-ScopeHealth.ps1` confirms once the account exists and is reachable.

`Get-AzAccessToken` / data-plane calls return 401 / 403

Symptom: `Connect-AzAccount` succeeds but a REST call to …`purview.azure.com` is unauthorized, or collections don't enumerate. **Cause:** No **data-plane role** at the collection scope (Collection admin / Data source admin / Data curator), or the wrong data-plane **resource URL** was used for the token. **Fix:** Assign the least-privilege data-plane role at the in-scope **collection** (Purview portal → **Data Map** → **Collections** → **Role assignments**, verify path). Confirm the token resource is the current Purview data-plane resource (`https://purview.azure.net` as of authoring — **verify in your tenant**). Re-run `DGLINE-ScopeHealth.ps1`. Control-plane (Azure RBAC on the account) is **not** the same as data-plane (collection) roles.

A REST call returns 404 / "api-version not supported"

Symptom: A scanning/catalog/lineage endpoint returns 404 or an api-version error. **Cause:** The `api-version` value or the endpoint path has changed — the Data Governance data-plane APIs evolve, and the classic vs. unified experiences expose different surfaces. **Fix:** Every `api-version` in the scripts is a **placeholder cited as of authoring** — override it with the `-…ApiVer` parameter using the **current** value from Microsoft's REST documentation for your tenant. Do not assume the path is wrong before checking the api-version.

Lineage isn't appearing after connecting ADF/Synapse/Power BI

Symptom: The integration source is connected to Purview but the asset's Lineage tab is empty. **Cause (most common):** Automatic lineage is captured only on **runs that happen after** the connection exists — a pipeline that ran **before** you connected it is **not back-filled**. Or the source/sink stores aren't **scanned**, so there's no asset to attach the edge to. Or there's simple processing delay. **Fix: Run the pipeline again** after connecting, and ensure both the source and sink stores are **registered and scanned** into the Data Map (Requirement 2.4). Re-run `DGLINE-Validate.ps1 -Asset '<sink qualified name>'`. Connect **early** — this is why the Requirements and Best Practices insist on it.

Custom-lineage push flags an asset as "not resolvable"

Symptom: `DGLINE-Configure.ps1 -WhatIf` prints `[WARN] input/output asset not resolvable in catalog`. **Cause:** The `InputQualifiedName` / `OutputQualifiedName` doesn't match a scanned asset — usually a hand-typed path, a casing/format difference, or the asset was never scanned. **Fix:** Use the **canonical qualified name from a scan** (copy it from the catalog / from `DGLINE-Validate.ps1` output), not a hand-built path. Scan the endpoint first if it isn't in the Data Map. Re-run `-WhatIf` until both ends resolve, then `-Commit`.

`DGLINE-Configure.ps1 -Commit` created lineage in the wrong place

Symptom: A custom-lineage `Process` points at the wrong asset. **Cause:** A wrong qualified name in the coverage-map CSV. **Fix:** The script printed the created **GUID** and the exact reverse `DELETE` call — run it to remove **only** that `Process` entity (it never deletes underlying data assets). Correct the CSV and re-push. This is why every push is dry-run (`-WhatIf`) and reviewed against `Data Lineage Coverage Map – DGLINE – Design` first.

Lineage renders but points at the wrong source ("confident wrong answer")

Symptom: The graph is present but a data owner says the upstream source or downstream consumer is wrong. **Cause:** The automatic connector captured a path you didn't expect, or a custom `Process` referenced the wrong endpoint — presence is not correctness. **Fix:** Walk the graph with the data owner (Testing Guide Test 7). Remove/repair the offending edge (delete the wrong custom `Process` by GUID, or correct the connection scope) and re-validate. Never sign off lineage on **presence** alone.

Coverage report shows many assets "without lineage"

Symptom: `DGLINE-Report.ps1` reports low coverage. **Cause:** Expected — assets legitimately have no lineage until a source is connected or custom lineage is pushed; or the endpoints aren't scanned; or the report's search scope is too broad (assets not actually in your traceability target). **Fix:** Treat each without-lineage asset as a **named follow-up** (scan the endpoint, connect the source, or push a custom `Process`). Narrow the report with `-Keyword`/`-Limit` to the in-scope estate so coverage is measured against the governance **target**, not the whole catalog. Coverage is closed **deliberately**, not assumed.

Scan shows failed / no runs for a source

Symptom: `DGLINE-Discovery.ps1` reports `no runs` / `failed` / `not readable` for a source. **Cause:** The source was registered but never scanned, the scan credential/integration runtime is misconfigured, or your role can't read the scan history. **Fix:** Run (or fix) the scan in the portal (**Data Map** → **Data sources** → **New scan**, verify path); confirm the scan credential and integration runtime; ensure you hold **Data source**

admin on the collection. Re-run `DGLINE-Discovery.ps1` and confirm a **completed** scan before expecting lineage.

When to stop and escalate

- The **Data Governance account/pricing model** or which **experience** (classic Data Map vs. unified

Unified Catalog) your tenant is on is unclear → verify against Microsoft's current documentation and your Azure subscription before provisioning or assuming an API surface; do not invent an endpoint.

- A determination is being asked of you — **which lineage coverage is *required*** for a regulatory/records

obligation, or **which source is *authoritative*** for a data element → escalate to data governance, records, privacy, or legal. This solution provides the graph and the coverage evidence, never the determination.

- A portal path or REST `api-version` in this guide **doesn't match your tenant** → this area is actively

evolving; verify the current path/version in your tenant before concluding the *step* is wrong.

7. Best Practices — Data Lineage Implementation (Microsoft Purview)

- **Connect the integration sources early — lineage is not retroactive.** Automatic lineage (ADF,

Synapse, Data Share, Power BI) is captured only on **runs after** the source is connected to Purview; historical runs are never back-filled. Connect first, then run — a pipeline that ran before the connection leaves a permanent hole.

- **Scan both ends before expecting an edge.** A lineage edge only renders when **both** the source and

the sink asset are known to the Data Map (scanned, or referenced by qualified name). "Connect the pipeline" and "scan the stores" are one task, not two — half of it produces a half-drawn graph.

- **Always dry-run custom lineage.** Run `DGLINE-Configure.ps1 -WhatIf` and read the `Process` payload —

the input/output qualified names — before `-Commit`; the script prints the created GUID and the exact reverse `DELETE` so removing a wrong `Process` is trivial and never touches the underlying data.

- **Prove both mechanisms on the pilot scope first.** Pick a pilot that exercises **one automatic** and

one custom path, so you validate the connector *and* the Atlas push before touching the estate. Lineage is cheap to fix on one path and expensive to unpick tenant-wide.

- **Measure coverage — a graph with silent holes is a liability.** Run `DGLINE-Report.ps1` and treat

every without-lineage asset as an explicit follow-up (missing scan, missing connection, or a `Process` to push). Trusting an incomplete lineage graph during an incident is worse than knowing it's incomplete.

- **Validate correctness, not just presence.** Have a data owner confirm the graph shows the **right**

sources and consumers. A lineage edge that renders but points at the wrong authoritative source is a confident wrong answer.

- **Reference assets by stable qualified names.** Custom-lineage `Process` entities reference their

inputs/outputs by **qualified name** — use the catalog's canonical qualified names (from a scan), not hand-typed paths, so the edge binds to the real asset and survives re-scans.

• **Keep data-plane roles least-privilege and collection-scoped.** Assign Data source admin / Data curator at the **collection** in scope, not root-collection admin for everyone; the Data Map role model is collection-scoped for a reason.

• **Treat portal paths and REST** `api-version` **values as provisional.** The Data Governance area is actively evolving (classic Data Map vs. the unified Unified Catalog experience) — verify each portal path and every `api-version` in your tenant against Microsoft's current docs rather than trusting a value from last quarter.

• **Deliver the capability; defer the determination.** What lineage coverage is *required* for a regulatory/records obligation, and which source is *authoritative* for a data element, are decisions for data governance, records, privacy, and compliance. You provide the graph and the measured coverage — they own the target.

8. Deliverables Summary — Data Lineage Implementation (Microsoft Purview)

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	Data lineage in the Purview Data Map / Unified Catalog, the capture model (automatic connectors vs. custom Atlas lineage), the Assess→Design→Configure→Pilot→Validate→Expand approach, limitations, and the naming convention
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing (Purview Data Governance account, data-plane roles, Az + REST toolchain, registered+scanned sources, connections, custom-lineage plan)
Global Requirements (Purview)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered portal + PowerShell/REST steps (assess the Data Map, design the coverage/connection map, scan endpoints, connect automatic sources pilot-first, push custom lineage `-WhatIf`→`-Commit`, validate the graph, measure coverage, expand)
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`DGLINE-Discovery.ps1`	Read-only discovery: enumerates registered **data sources**, their **last-scan status**, and per source whether it is **lineage-capable/connected** (ADF/Synapse/Data Share/Power BI) or **needs custom lineage**. Writes `Data Lineage Source Inventory – DGLINE – Discovery`
`DGLINE-Configure.ps1`	Custom-lineage configuration/remediation helper: pushes Atlas `Process` entities (inputs/outputs by qualified name) to the catalog data-plane API from a coverage-map CSV; `-WhatIf` **by default**, `-Commit` to apply; prints the created GUID and the reverse `DELETE` for each push
`DGLINE-Validate.ps1`	Read-only lineage validation: resolves an asset by qualified name and queries the **Atlas lineage endpoint**, reporting the **upstream/downstream** entities and edge count for a named asset
`DGLINE-Report.ps1`	Read-only coverage report: enumerates in-scope assets and reports **with-lineage vs. without-lineage** per collection/source plus last-scan status. Writes `Data Lineage Coverage – DGLINE – Report`
`DGLINE-ScopeHealth.ps1`	Read-only scope/health helper: confirms the account endpoint is reachable, acquires a `purview.azure.net` data-plane token, and lists the **collections** the token can enumerate

All scripts are **safe by default** — read-only, or a `-WhatIf` preview requiring an explicit `-Commit` to change anything. No credentials or secrets are stored in code; every script uses interactive modern-auth (`Connect-AzAccount`) and a short-lived Azure AD **data-plane token** (`Get-AzAccessToken -ResourceUrl 'https://purview.azure.net'`). REST `api-version` values and portal paths are cited as of authoring — **verify against your tenant.**

What the buyer receives

- This implementation guide as a **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process following the model: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.**

Outcome

Data lineage implemented in Microsoft Purview — in-scope sources registered and scanned, automatic lineage flowing from the connected integration systems, custom lineage pushed for the systems that don't self-report, the graph validated for correctness on a pilot scope, and **coverage measured** against the governance target — so that, for the in-scope estate, data can be traced from source to report and back, defensibly. Regulatory/records determinations and the authoritative-source designation are deferred to the data-governance, records, privacy, and legal functions.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

DGLINE-Configure.ps1

Purpose: For source->report paths not covered by an automatic connector, creates CUSTOM lineage by pushing Apache Atlas "Process" entities (inputs/outputs referenced by qualified name) to the Purview catalog data-plane API, from a coverage-map CSV. SAFE BY DEFAULT: -WhatIf previews the exact payload and makes no changes; -Commit applies. Prints the created GUID and the reverse DELETE for rollback. Stores no secrets. HOW TO RUN Preview (default, no changes): powershell -File .\DGLINE-Configure.ps1 -Account <acct> -MapPath .\LineageMap.csv -WhatIf Apply (pilot path first): powershell -File .\DGLINE-Configure.ps1 -Account <acct> -MapPath .\LineageMap.csv -Commit MapPath CSV columns: ProcessName,ProcessType,InputQualifiedName,InputType,OutputQualifiedName,OutputType PowerShell 5.1+ or 7+. Requires Az.Accounts. Data curator role on the in-scope collection. EXPECTED RESULTS -WhatIf prints one Atlas Process payload per row and flags unresolved assets, changing nothing. -Commit creates each Process and prints its GUID + the reverse DELETE. PASS = pilot lineage renders (validate with DGLINE-Validate.ps1) and can be removed via the printed reverse command. .NOTES Purview — Data Lineage Implementation (DGLINE). Write-gated by -Commit; -WhatIf default. No secrets in code. Atlas entity/lineage endpoints + api-version are cited as of authoring and evolving — verify in your tenant. Never deletes underlying data assets — only the Process (lineage) entities this script created. > [CmdletBinding(SupportsShouldProcess, ConfirmImpact='High')] param([Parameter(Mandatory)][string]$Account, [Parameter(Mandatory)][string]$MapPath, [switch]$Commit, [string]$ResourceUrl = 'https://purview.azure.net', [string]$CatalogApiVer = '2022-03-01-preview', [string]$ReportName = 'Data Lineage Connections - DGLINE - Configure')

How to run: Preview (default, no changes): powershell -File .\DGLINE-Configure.ps1 -Account <acct> -MapPath .\LineageMap.csv -WhatIf Apply (pilot path first): powershell -File .\DGLINE-Configure.ps1 -Account <acct> -MapPath .\LineageMap.csv -Commit MapPath CSV columns: ProcessName,ProcessType,InputQualifiedName,InputType,OutputQualifiedName,OutputType PowerShell 5.1+ or 7+. Requires Az.Accounts. Data curator role on the in-scope collection. EXPECTED RESULTS -WhatIf prints one Atlas Process payload per row and flags unresolved assets, changing nothing. -Commit creates each Process and prints its GUID + the reverse DELETE. PASS = pilot lineage renders (validate with DGLINE-Validate.ps1) and can be removed via the printed reverse command. .NOTES Purview — Data Lineage Implementation (DGLINE). Write-gated by -Commit; -WhatIf default. No secrets in code. Atlas entity/lineage endpoints + api-version are cited as of authoring and evolving — verify in your tenant. Never deletes underlying data assets — only the Process (lineage) entities this script created. > [CmdletBinding(SupportsShouldProcess, ConfirmImpact='High')] param([Parameter(Mandatory)][string]$Account, [Parameter(Mandatory)][string]$MapPath, [switch]$Commit, [string]$ResourceUrl = 'https://purview.azure.net', [string]$CatalogApiVer = '2022-03-01-preview', [string]$ReportName = 'Data Lineage Connections - DGLINE - Configure')

DGLINE-Discovery.ps1

Purpose: Enumerates the registered data sources in the Microsoft Purview Data Map, reports each source's last-scan status, and flags — per source — whether it produces lineage AUTOMATICALLY (Azure Data Factory, Synapse, Data Share, Power BI) or will need CUSTOM lineage pushed via the Atlas

API. Read-only. Makes no changes. Writes the export "Data Lineage Source Inventory - DGLINE - Discovery". HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGLINE-Discovery.ps1 -Account <purview-account-name> -CsvPath .\LineageDiscovery.csv PowerShell 5.1+ or 7+. Requires Az.Accounts (Connect-AzAccount). Uses the Purview data-plane scanning API. Read data-plane role (Data source admin / Data reader) on the in-scope collections. EXPECTED RESULTS A table of sources with kind, last-scan status, and lineage capability, plus an optional CSV. PASS = every in-scope source is listed as "registered + scanned" with automatic/custom flagged. .NOTES Purview — Data Lineage Implementation (DGLINE). Read-only. No secrets in code. Data lineage is a Data Map / Unified Catalog capability; source kinds, portal paths, and REST api-version values are evolving — verify against your tenant. api-version below is a placeholder. > [CmdletBinding()] param([Parameter(Mandatory)][string]$Account, [string]$CsvPath, [string]$ResourceUrl = 'https://purview.azure.net', [string]$ScanApiVer = '2022-02-01-preview', [string]$ReportName = 'Data Lineage Source Inventory - DGLINE - Discovery')

How to run: powershell -ExecutionPolicy Bypass -File .\DGLINE-Discovery.ps1 -Account <purview-account-name> -CsvPath .\LineageDiscovery.csv PowerShell 5.1+ or 7+. Requires Az.Accounts (Connect-AzAccount). Uses the Purview data-plane scanning API. Read data-plane role (Data source admin / Data reader) on the in-scope collections. EXPECTED RESULTS A table of sources with kind, last-scan status, and lineage capability, plus an optional CSV. PASS = every in-scope source is listed as "registered + scanned" with automatic/custom flagged. .NOTES Purview — Data Lineage Implementation (DGLINE). Read-only. No secrets in code. Data lineage is a Data Map / Unified Catalog capability; source kinds, portal paths, and REST api-version values are evolving — verify against your tenant. api-version below is a placeholder. > [CmdletBinding()] param([Parameter(Mandatory)][string]$Account, [string]$CsvPath, [string]$ResourceUrl = 'https://purview.azure.net', [string]$ScanApiVer = '2022-02-01-preview', [string]$ReportName = 'Data Lineage Source Inventory - DGLINE - Discovery')

DGLINE-Report.ps1

Purpose: Enumerates the in-scope catalog assets (optionally filtered by keyword/collection) and reports lineage COVERAGE — how many assets have at least one lineage edge vs. none — so gaps are explicit and can be closed deliberately (missing scan, missing connection, or a custom Process to push). Read-only. Makes no changes. Writes "Data Lineage Coverage - DGLINE - Report". HOW TO RUN powershell -File .\DGLINE-Report.ps1 -Account <acct> [-Keyword '<filter>'] [-Limit 200] -CsvPath .\LineageCoverage.csv PowerShell 5.1+ or 7+. Requires Az.Accounts. Data curator/reader role on the in-scope collection. EXPECTED RESULTS A per-asset table (asset, type, has-lineage) and a summary (with-lineage / without-lineage counts), plus an optional CSV. PASS = a dated coverage snapshot exists and each without-lineage asset is a known follow-up item. .NOTES Purview — Data Lineage Implementation (DGLINE). Read-only. No secrets in code. Catalog search + Atlas lineage endpoints + api-version are cited as of authoring and evolving — verify in your tenant. Coverage is measured against the governance team's target; they own which paths MUST be traceable. > [CmdletBinding()] param([Parameter(Mandatory)][string]$Account, [string]$Keyword = '*', [int]$Limit = 200, [string]$CsvPath, [string]$ResourceUrl = 'https://purview.azure.net', [string]$CatalogApiVer = '2022-03-01-preview', [string]$ReportName = 'Data Lineage Coverage - DGLINE - Report')

How to run: powershell -File .\DGLINE-Report.ps1 -Account <acct> [-Keyword '<filter>'] [-Limit 200] -CsvPath .\LineageCoverage.csv PowerShell 5.1+ or 7+. Requires Az.Accounts. Data curator/reader role on the in-scope collection. EXPECTED RESULTS A per-asset table (asset, type, has-lineage) and a summary (with-lineage / without-lineage counts), plus an optional CSV. PASS = a dated coverage snapshot exists and each without-lineage asset is a known follow-up item. .NOTES Purview — Data Lineage Implementation (DGLINE). Read-only. No secrets in code. Catalog search + Atlas lineage

endpoints + api-version are cited as of authoring and evolving — verify in your tenant. Coverage is measured against the governance team's target; they own which paths MUST be traceable. > [CmdletBinding()] param([Parameter(Mandatory)][string]$Account, [string]$Keyword = '*', [int]$Limit = 200, [string]$CsvPath, [string]$ResourceUrl = 'https://purview.azure.net', [string]$CatalogApiVer = '2022-03-01-preview', [string]$ReportName = 'Data Lineage Coverage - DGLINE - Report')

DGLINE-ScopeHealth.ps1

Purpose: Confirms the Microsoft Purview (Data Governance) account is reachable at its data-plane endpoint, acquires a short-lived Azure AD data-plane token, and lists the collections the signed-in identity can enumerate — establishing you hold the least-privilege data-plane roles before doing lineage work. Read-only. Makes no changes. Stores no secrets. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGLINE-ScopeHealth.ps1 -Account <purview-account-name> PowerShell 5.1+ or 7+. Requires Az.Accounts (Connect-AzAccount). Uses the Purview data-plane resource https://purview.azure.net. Data-plane roles: Collection admin / Data source admin / Data curator. EXPECTED RESULTS The account endpoint responds, a data-plane token is acquired, and the enumerable collections are listed. PASS = endpoint reachable + token acquired + at least the in-scope collection listed. .NOTES Purview — Data Lineage Implementation (DGLINE). Read-only. No secrets in code. Data lineage is a Data Map / Unified Catalog capability; provisioning, roles, portal paths, and REST api-version values are evolving — verify against your tenant. api-version values below are placeholders. > [CmdletBinding()] param([Parameter(Mandatory)][string]$Account, Verify the current data-plane resource URL and api-version in Microsoft's documentation for your tenant. [string]$ResourceUrl = 'https://purview.azure.net', [string]$AccountApiVer = '2019-11-01-preview')

How to run: powershell -ExecutionPolicy Bypass -File .\DGLINE-ScopeHealth.ps1 -Account <purview-account-name> PowerShell 5.1+ or 7+. Requires Az.Accounts (Connect-AzAccount). Uses the Purview data-plane resource https://purview.azure.net. Data-plane roles: Collection admin / Data source admin / Data curator. EXPECTED RESULTS The account endpoint responds, a data-plane token is acquired, and the enumerable collections are listed. PASS = endpoint reachable + token acquired + at least the in-scope collection listed. .NOTES Purview — Data Lineage Implementation (DGLINE). Read-only. No secrets in code. Data lineage is a Data Map / Unified Catalog capability; provisioning, roles, portal paths, and REST api-version values are evolving — verify against your tenant. api-version values below are placeholders. > [CmdletBinding()] param([Parameter(Mandatory)][string]$Account, Verify the current data-plane resource URL and api-version in Microsoft's documentation for your tenant. [string]$ResourceUrl = 'https://purview.azure.net', [string]$AccountApiVer = '2019-11-01-preview')

DGLINE-Validate.ps1

Purpose: Resolves an asset by qualified name and queries the Purview catalog Atlas LINEAGE endpoint, reporting the upstream and downstream related entities and the edge count for that asset — so you can confirm a lineage path renders (and is correct) after connecting a source or pushing custom lineage. Read-only. Makes no changes. Stores no secrets. HOW TO RUN powershell -File .\DGLINE-Validate.ps1 -Account <acct> -Asset '<qualified-name>' [-Depth 3] PowerShell 5.1+ or 7+. Requires Az.Accounts. Data curator/reader role on the in-scope collection. EXPECTED RESULTS The asset's GUID, plus tables of upstream and downstream related entities and the total edge count. PASS = the expected upstream source(s) and downstream consumer(s) are listed (verify correctness with a data owner — presence alone is not correctness). .NOTES Purview — Data Lineage Implementation (DGLINE). Read-only. No secrets in code. Catalog search + Atlas lineage endpoints + api-version are cited as of authoring and evolving — verify in your tenant. > [CmdletBinding()] param([Parameter(Mandatory)][string]$Account,

[Parameter(Mandatory)][string]$Asset, [int]$Depth = 3, [string]$ResourceUrl = 'https://purview.azure.net', [string]$CatalogApiVer = '2022-03-01-preview')

How to run: powershell -File .\DGLINE-Validate.ps1 -Account <acct> -Asset '<qualified-name>' [-Depth 3] PowerShell 5.1+ or 7+. Requires Az.Accounts. Data curator/reader role on the in-scope collection. EXPECTED RESULTS The asset's GUID, plus tables of upstream and downstream related entities and the total edge count. PASS = the expected upstream source(s) and downstream consumer(s) are listed (verify correctness with a data owner — presence alone is not correctness). .NOTES Purview — Data Lineage Implementation (DGLINE). Read-only. No secrets in code. Catalog search + Atlas lineage endpoints + api-version are cited as of authoring and evolving — verify in your tenant. > [CmdletBinding()] param([Parameter(Mandatory)][string]$Account, [Parameter(Mandatory)][string]$Asset, [int]$Depth = 3, [string]$ResourceUrl = 'https://purview.azure.net', [string]$CatalogApiVer = '2022-03-01-preview')